

# Module 8

I<sup>2</sup>C — UVM+SV Verification

# Learning objectives

- I<sup>2</sup>C UVM agent: I2cTransaction, I2cSequence, I2cDriver, I2cMonitor, I2cScoreboard.
- Interface: i2c\_if (clk, rst\_n, start, addr, data\_in, scl, sda, clk\_div\_tick, busy, done) connects U...
- How to hook up a UVM monitor and scoreboard to a bus-oriented DUT.

# Prerequisites

- Module 7 (I<sup>2</sup>C protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM\_HOME or vendored UVM).

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["I<sup>2</sup>C protocol recap"]

T2["Toolchain"]

T1 --> T2

T2 --> N["Next module in course"]

Extend I<sup>2</sup>C verification to **\*\*UVM+SV\*\*** — I<sup>2</sup>C agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

# Overview

- Module 8 builds on Module 7 (I<sup>2</sup>C protocol + RTL + basic testbench):

# How to learn this module

# Suggested learning path

- Read the detailed I<sup>2</sup>C learning guide:  
I2C\_LEARNING\_GUIDE.md — what I<sup>2</sup>C is, how it works  
(START/STOP...
- Read the protocols + UVM overview:  
LEARNING\_GUIDE\_PROTOCOLS\_AND\_UVM.md — Part B § 5.  
How UVM Maps t...

# Design architecture

# 1. I<sup>2</sup>C DUT (reused from Module 7)

- i2c\_master + clk\_div; i2c\_if for UVM connection.
- Wire sequence: START → address+W → data → STOP.

## Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph dut["I2C DUT (Module 7)"]
    DIV["clk_div"]
    M["i2c_master"]
    DIV --> M
  end
end
```

## 2. UVM agent

- I2cTransaction holds `addr` + `data`; monitor fills `observed\_\*`.
- I2cMonitor reimplements baseline bus sampling inside UVM.
- I2cScoreboard checks address and data phases independently.

### Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence\naddr + data"] --> DRV["I2cDriver"]

DRV --> DUT["I2c\_master"]

DUT --> MON["I2cMonitor\nSTART + SCL/SDA sample"]

MON --> SB["I2cScoreboard\naddr and data phases"]

# 3. Course capstone

- Compare UART (async), SPI (SCLK+CS), I<sup>2</sup>C (shared bus) — same UVM skeleton, different monitors.

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence"] --> DRV["I2cDriver"]

DRV --> DUT["I2c\_master"]

DUT --> MON["I2cMonitor"]

MON --> SBI["addr + data\scoreboard"]

# RTL block diagram (reference)

## Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["I<sup>2</sup>C DUT (Module 7)"]

DIV["clk\_div"]

M["i2c\_master"]

DIV --> M

end

Module 8: DUT hierarchy and signal flow.

# Verification / testbench diagram (reference)

## Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence\naddr + data"] --> DRV["I2cDriver"]

DRV --> DUT["i2c\_master"]

DUT --> MON["I2cMonitor\nSTART + SCL/SDA sample"]

MON --> SB["I2cScoreboard\naddr and data phases"]

Module 8: stimulus, observation, and checking.

# I2cTransaction & agent

```
/**
 * Module 4: I2C master UVM test (simplified, single write
 transaction).
 *
 * Teaching example:
 * - Simplified I2C master (no real open-drain or ACK handling).
 * - UVM agent requests address+data writes and a monitor reconstructs
 them
 *   from SDA/SCL activity under a single START/STOP frame.
 */

`include "uvm_macros.svh"
import uvm_pkg::*;

`timescale 1ns/1ps

//
```

# **Execution & simulation flow**

# How the example runs (toolchain)

- Makefile: Verilator + UVM\_HOME compile RTL, interface, and test\_\*.sv
- UVM phases: build → connect → run\_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test\_\* (see module8 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

# Example directory layout

- dut/: i2c\_master.v, clk\_div.v (same as Module 7)
- test\_i2c\_uvm.sv: Interface, I2cTransaction, I2cSequence, I2cDriver, I2cMonitor, I2cScoreboard, I2cA...

# Makefile — UVM I2C sim

```
# Makefile for Module 8 I2C UVM test
#
# Usage:
#   make SIM=verilator TEST=test_i2c_uvm
#

SIM ?= verilator
TEST ?= test_i2c_uvm

# Prefer externally-set UVM_HOME, but fall back to vendored UVM.
# From module8/examples/i2c_uvm, repo root is ../../..
UVM_HOME ?= \$(shell echo \${UVM_HOME})
UVM_HOME_FALLBACK := ../../../../tools/uvm-2017/1800.2-2017-1.0
UVM_HOME_FALLBACK_ALT :=
../../../../tools/learn_uvm2017_sv_verilator/tools/uvm-2017/1800.2-2017-1.0
```

# **Key files to study**

# Open these in the repo

- ``module8/examples/i2c_uvm/dut/i2c_master.v`` — reused from Module 7
- ``module8/examples/i2c_uvm/test_i2c_uvm.sv`` — I2cMonitor + two-field scoreboard
- ``docs/LEARNING_GUIDE_PROTOCOLS_AND_UVM.md`` — § 8 I<sup>2</sup>C UVM mapping

# **Verification & testing methods**

# 1. Two-phase scoreboard

- Separate checks for address+W byte and data byte — catches shifted or merged fields.

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence"] --> DRV["I2cDriver"]

DRV --> DUT["I2c\_master"]

DUT --> MON["I2cMonitor"]

MON --> SBI["addr + data\scoreboard"]

## 2. Reuse baseline monitor logic

- Module 7 inline monitor → Module 8 I2cMonitor for scalable tests.

### Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence"] --> DRV["I2cDriver"]

DRV --> DUT["I2c\_master"]

DUT --> MON["I2cMonitor"]

MON --> SBI["addr + data\nscoreboard"]

# 3. Extensions

- ACK/NACK, reads, multi-master —  
[LEARNING\_GUIDE § 8]  
(LEARNING\_GUIDE § 8)  
IDE\_PROTOCOLS\_AND\_UVM.md#8-i2c-  
uvm-...

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence"] --> DRV["I2cDriver"]

DRV --> DUT["I2c\_master"]

DUT --> MON["I2cMonitor"]

MON --> SBI["addr + data\scoreboard"]

# Syllabus topics

# 1. I<sup>2</sup>C protocol recap

- START/STOP on SCL high; MSB-first address and data on SDA.

## 2. Toolchain

- ``make SIM=verilator TEST=test_i2c_uvm`;`  
  ``+UVM_TESTNAME=test_i2c_uvm`.`

# Hands-on examples

# Module 8 self-check

```
$ ./scripts/module8.sh --help
```

Terminal: module8\_check

```
[[0;36m===== [[0m
[[0;36mModule 8: Demo commands (I²C UVM example) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the I²C UVM example (inside the example directory):
cd module8/examples/i2c_uvm
make SIM=verilator TEST=test_i2c_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module8/EXAMPLES.md and module8/examples/i2c_uvm/README.md for more details.
```

# Example 1: I<sup>2</sup>C UVM agent

- I<sup>2</sup>C DUT from Module 7 with UVM agent and two-phase scoreboard (addr + data)
- `I2cMonitor` reuses baseline bus-sampling ideas in reusable form
- Course capstone: compare UART, SPI, and I<sup>2</sup>C verification patterns

# Demo: I<sup>2</sup>C UVM agent

```
$ cd module8/examples/i2c_uvm && make SIM=verilator TEST=test_i2c_uvm
```

Terminal: ex01\_i2c\_uvm

## Success

You should see UVM phases, DRIVER/MONITOR/SCOREBOARD messages, and a scoreboard summary with all wri

# Key concepts

# 1. Two-phase scoreboard

- Compare ``observed_addr`` and ``observed_data`` separately from one transaction.

## 2. Course capstone

- Same UVM skeleton as UART/SPI; only monitor protocol rules differ.

# Common pitfalls

# Merging address and data checks

- Mistake: One expected byte for the whole transaction.
- Correct: Separate expected/observed for address+W and data phases.
- Why: Off-by-one bit in address can hide behind a correct data byte.

# Skipping baseline Module 7

- Mistake: Jumping to UVM without understanding inline bus monitor logic.
- Correct: Run i2c\_baseline first; then map monitor to I2cMonitor.
- Why: UVM monitor is the same idea in a reusable component.

# Practice & assessment

# What you should know

- Describe the I<sup>2</sup>C UVM agent and how I2cMonitor maps to SCL/SDA behavior.
- Explain two-phase scoreboard checking (address vs data).
- Run ``i2c_uvm`` and interpret UVM / scoreboard output.
- Compare verification architecture across UART, SPI, and I<sup>2</sup>C modules in this course.

# Exercises

- Run i2c\_uvm
- Trace one transaction
- Change the sequence
- Compare across protocols

# Assessment checklist

- Can describe the I<sup>2</sup>C UVM agent and how it maps to I<sup>2</sup>C (addr+data, monitor on SCL/SDA).
- Can run i2c\_uvm and interpret UVM output.
- Can compare UVM structure across UART, SPI, and I<sup>2</sup>C.

# Summary & next steps

- Extend I<sup>2</sup>C verification to **\*\*UVM+SV\*\*** — I<sup>2</sup>C agent  
(transaction, sequence, driver, monitor, scoreboa...
- Complete module8/CHECKLIST.md
- Review module8/EXAMPLES.md and run each lab
- Next: Next module in course